

DigiPen Institute of Technology Singapore
AI Application Development Bootcamp · 2026 Summer

CAPSTONE PROJECT
Requirements & Guidelines

Capstone Presentation: Day 14

1. Overview

The Capstone Project is the culminating assessment of the AI Application Development Bootcamp. It requires you to independently design, build, document, and present a practical AI-powered application that demonstrates the technical and professional skills you have developed across all five phases of the programme.

The capstone is not a theoretical exercise. You are expected to deliver working software with real API integration, a clear use case, meaningful output quality, and professional documentation and presentation.

Why the capstone matters

The capstone project is the primary piece of portfolio evidence you will take out of this bootcamp. It demonstrates to future employers that you can not only use AI tools, but that you can build something useful, think through application design, handle errors and edge cases, and communicate your work clearly. Employers hiring junior developers with AI skills will ask: “Show me something you built.” This is your answer.

2. AI APIs

You are not required to pay for any service. The APIs listed below all have free tiers that are more than sufficient for a capstone project. You only need one.

API	Free Access Details
Google Gemini	Available free via Google AI Studio at ai.google.dev . No credit card required. This is the recommended starting point for most students.
Groq	Free at console.groq.com . Runs open-source models including LLaMA and Mixtral. Very fast response times. No credit card required.
Ollama (local)	Runs AI models directly on your own laptop at no cost. No internet connection needed during use. Setup instructions are on Moodle.
OpenAI	Requires a small credit top-up (approximately USD \$5). Fully optional — Google Gemini or Groq will serve the same purpose for free.
Anthropic Claude	Requires a billing account. Fully optional. Use Google Gemini or Groq if you prefer to avoid any cost.

3. Project Requirements

Your capstone project must meet all of the following requirements to be considered complete. Requirements apply regardless of which project idea you choose.

3.1 The Application

Your application must do all of the following:

1. **Connect to a free AI API.** Use Google Gemini, Groq, Ollama, or another free provider.
2. **Accept user input.** The user types or provides something, and your application returns a meaningful AI-generated response.
3. **Have a specific purpose.** Your app must do something defined and useful. A general chatbot with no focus is not sufficient.
4. **Include a designed system prompt.** Write a system prompt that shapes the AI's behaviour for your use case. Store it as a named variable in your code, not as an unnamed string buried inside a function.
5. **Handle errors.** If an API call fails or returns nothing, display a clear message. Do not let the application crash with an unhandled error or exception.
6. **Run cleanly from your README.** Another person must be able to follow your setup instructions and run the application without needing to ask you anything.

3.2 Your Code

Good code is readable code. These are the standards expected:

- You may use any programming language. Choose the language you are most confident with — Python, JavaScript, C++, or another language are all acceptable. If you use Python, use version 3.10 or later.
- **Keep your API key out of your source code.** For Python and JavaScript, use a .env file with a dotenv library. For C++, use a separate config file or environment variable and add it to .gitignore. Your key must never appear in your source code or be committed to GitHub.
- **Include a dependency file** listing the libraries or packages your project needs. For Python projects, this is requirements.txt (e.g. google-generativeai, python-dotenv). For JavaScript projects, this is package.json. For C++ projects, document your dependencies in the README.
- **Each function should have a short comment** explaining what it does. One sentence per function is enough.
- Your system prompt should be a named constant at the top of your file, so it is easy to find and easy to change.

3.3 Your GitHub Repository

Your project must be in a public GitHub repository under your own account. The repository must contain these four files:

- **README.md** — following the structure in Section 4.
- **.env.example** — showing which environment variables or config values are needed, with placeholder values instead of real keys. For C++ projects without dotenv support, include a config.example file with the same purpose.
- **Dependency file** — listing all dependencies (requirements.txt for Python, package.json for JavaScript, or equivalent for your language).
- **All source code files.**

The following must never be included in the repository:

- **Your .env file** or any real API keys. Add .env to your .gitignore before you make your first commit.
- Any personal, confidential, or sensitive data.

Try to commit your code regularly as you work, rather than pushing everything in one go at the end. A commit history that shows progress over time is a good professional habit and something the panel will notice.

3.4 The Presentation

Presentations take place on Day 14. Each student has 10 minutes to present and demonstrate, followed by 5 minutes of questions from the panel.

Component	What is expected
Slides	4-5 slides plus a live demo. See Section 5 for the structure. Submit your deck to Moodle by end of Day 13.
Live demo	Run your application live and demonstrate at least two different inputs. Walk the panel through what is happening.
Q&A	Be ready to explain your system prompt, your error handling, and at least one design decision you made. The panel will look at your code, not just the output.
Backup	Bring a short screen recording of your application running. You may not need it, but it is good to have in case of any connectivity issues.

4. README Structure

Your README.md must cover the following seven sections. It does not need to be long — clear and complete is what matters.

Section 1 — Project Title and Description

Name the project and describe what it does in one to two sentences. State who it is for.

Section 2 — Problem Statement

Describe the problem your application addresses and why it is useful. Two to three sentences is sufficient.

Section 3 — Technology Stack

List the language, libraries, and AI API you used. For example: Python, google-generativeai, python-dotenv, Google Gemini API — or JavaScript, Node.js, Gemini API — or C++ with libcurl and the Groq REST API.

Section 4 — Setup Instructions

Write numbered steps that allow someone to clone and run the project from scratch. For example:

7. Clone the repository.
8. Install dependencies (e.g. `pip install -r requirements.txt` for Python, or `npm install` for JavaScript, or document and follow the build steps described in your README for other languages).
9. Copy `.env.example` to `.env` and fill in your API key.
10. Run the application (e.g. `python main.py`, or `node index.js`, or `./build/myapp` — use whatever command suits your language).

Section 5 — Usage Examples

Show at least two examples of a user input and the application's output.

Section 6 — Known Limitations

Be honest about what the application does not handle well. Two examples are enough.

Section 7 — Future Improvements

Note one or two ways you would extend the project given more time.

5. Presentation Slide Structure

Presentation slides plus a live demo. The live demonstration is the most important part of the presentation — the slides exist to frame it.

Slide	Content
1 — Title	Project name, your name, and one sentence describing what the application does.
2 — The Problem	What problem does your application address? Who would use it, and why is AI the right tool for this?
3 — How It Works	A simple diagram or a few bullet points showing the flow: user input → your code → AI API → output. No detailed architecture is needed.
Live Demo (no slide)	Switch directly to your application. Run it live, show at least two different inputs, and explain what is happening as you go. No slide is needed for this section.
5 — What You Learned	One thing that was harder than expected and how you worked through it. One thing you would do differently. One idea for what comes next.

6. Assessment

The Capstone Project is assessed as Completed or Not Completed. There are no letter grades. The table below describes what a Completed result looks like across each area.

Area	What Completed looks like
Working Application	Runs end-to-end without errors. Takes user input and returns a meaningful AI-generated response.
API Integration	A free AI API is connected and working. The API key is in a .env file and nowhere in the source code.
System Prompt	The application has a deliberately written system prompt, stored as a named variable, that shapes the AI's behaviour for the chosen use case.
Error Handling	API failures or empty responses are caught and communicated to the user. The application does not crash with an unhandled exception.
README	All seven sections are present. A person who has not seen the project before can follow the instructions and run it successfully.
GitHub Repository	Public repository containing all required files. No API keys or .env files committed. Commit history shows development over time.
Presentation	Application is demonstrated live. The student explains design decisions clearly and can discuss the code in the Q&A.

Minimum threshold:

- The application runs without errors.
- The user can provide input and receive an AI-generated response.
- You can explain what your system prompt does and why you wrote it the way you did.

If your project meets these three criteria, you are on track for a Completed result.

7. Key Dates

The milestones below apply to all students. Slide deck submission is via Moodle. All other work is submitted through GitHub or shown during lab check-ins.

Milestone	Details
Day 13 — GitHub Submission	Push your final code to GitHub. The README and .env.example must be in place.
Day 13 — Slide Deck	Submit your presentation slide deck to Moodle by end of day.
Day 14 — Presentation	Present and demonstrate your application to the panel. 10 minutes plus 5 minutes Q&A.

8. Sample Capstone Project Ideas

The following ideas are provided as inspiration. You are not required to choose one of these — original ideas are welcome and encouraged. All ideas below are scoped to be achievable within the bootcamp timeline and aligned with the curriculum and learning outcomes.

Idea 1 Singapore Hawker Food Recommendation Chatbot

A conversational chatbot that recommends hawker dishes and food stalls based on what the user is craving, their MRT location, and any dietary preferences. The bot remembers what the user has said earlier in the session.

Free API: Google Gemini (free)

Skills practised:

- Writing a system prompt with a defined personality and consistent tone
- Building multi-turn conversation memory using a messages history list
- Handling vague or ambiguous requests from the user
- Formatting responses consistently across different questions

If you want to go further: Add a Streamlit chat interface with a sidebar showing the conversation history.

Idea 2 Resume and Cover Letter Assistant

The user provides a job description and their resume. The application identifies skill gaps and generates a tailored cover letter draft. The user can then ask for revisions through follow-up messages.

Free API: Google Gemini (free) or Groq (free)

Skills practised:

- Designing a multi-step prompt pipeline: analyse → identify gaps → generate draft
- Handling long combined inputs reliably
- Structuring the output into clear, labelled sections
- Allowing the user to refine the result through follow-up turns

If you want to go further: Save the generated cover letter to a .txt file automatically.

Idea 3 Personal Study Tutor

The user picks a subject and difficulty level. The tutor explains concepts, generates quiz questions, evaluates answers, and adjusts its approach based on how the student is doing. Everything is tracked through the conversation history.

Free API: Google Gemini (free) or Groq (free)

Skills practised:

- Creating a tutor persona through the system prompt
- Using conversation history to avoid repeating covered material
- Generating quiz questions and evaluating free-text answers
- Adapting difficulty in response to the student's performance

If you want to go further: Generate a session summary at the end, listing topics covered and areas to revisit.

Idea 4 Code Explainer

Accepts code snippets in any programming language from the user and provides structured feedback: correctness analysis, style suggestions, potential bugs, edge case identification, and a refactored version. Useful for beginner-to-intermediate programmers in any language.

Skills practised:

- Designing a system prompt that enforces structured, plain-language output
- Tuning temperature for consistent, analytical responses
- Structuring the output into labelled sections: explanation, issues, suggestion
- Handling edge cases such as empty input or unrecognisable code

If you want to go further: Let the user ask follow-up questions about specific lines of the explanation.

Idea 5 Text Adventure Game with an AI Game Master

A text-based game where the AI narrates the story and responds to the player's actions. Location, inventory, and health are tracked across turns through conversation history. The entire game world is defined through the system prompt.

Free API: Google Gemini (free) or Groq (free)

Skills practised:

- Defining a complete game world and its rules through the system prompt
- Tracking game state in the conversation history across many turns
- Keeping the narrative consistent and on-theme throughout a session
- Handling unexpected or out-of-scope player inputs without breaking the experience

If you want to go further: Let the player choose from two or three different game worlds at the start.

Idea 6 Document Q&A Tool

The user provides a text document — meeting notes, an article, study material — and then asks questions about it. The application answers only from the document's content and clearly says so when the answer is not there.

Free API: Google Gemini (free) — supports long context natively

Skills practised:

- Injecting document content into the prompt as context
- Constraining the AI to answer only from the provided document
- Handling documents that are longer than a single context window
- Supporting multiple questions about the same document in one session

If you want to go further: Add support for PDF and Markdown files in addition to plain text.

Idea 7 UX Research Synthesis Tool

The user pastes raw qualitative data — interview notes, survey responses — and receives a structured summary identifying key themes, pain points, and user needs. Particularly well suited for UXGD students.

Free API: Google Gemini (free) or Groq (free)

Skills practised:

- Designing a structured output format with consistent section headings
- Handling long and unstructured input text
- Instructing the AI to flag uncertainty rather than fabricate conclusions
- Allowing follow-up questions on specific themes from the summary

If you want to go further: Export the structured summary as a formatted Markdown or plain-text file.

Idea 8 Your Own Idea

You are free to build anything that meets the requirements in Section 3. A focused application that does one thing well is a stronger submission than a broad one that only partially works. Confirm your idea with your instructor before you start building.

Free API: Any free API

Skills practised:

- Must connect to at least one free AI API
- Must address a specific, clearly defined use case
- Must include a deliberately designed system prompt
- Must run without errors and be demonstrable within your 10-minute presentation slot

If you want to go further: Once the core is working, add one feature that goes beyond the basic requirements.

9. Submission Checklist

Work through this list before Day 14. Every item is required.

Done	Item
☐	The application runs end-to-end without errors.
☐	The AI API is connected and returning responses.
☐	The API key is in a .env file and does not appear in the source code.
☐	The system prompt is defined as a named variable.
☐	API failures are handled — the application displays a message rather than crashing.
☐	The README contains all seven sections and the setup instructions work.
☐	The GitHub repository is public.

☐	The .env.example file is in the repository with placeholder values.
☐	The dependency file is in the repository (requirements.txt, package.json, or equivalent for your language).
☐	The .env file is in .gitignore and has not been committed.
☐	The repository has more than one commit and the history shows progress over time.
☐	The slide deck has been submitted to Moodle by end of Day 13.
☐	The live demo has been practised with at least two prepared input scenarios.
☐	You can explain your system prompt and the reasoning behind it.

10. Support

Reference Documentation

- Google Gemini API — ai.google.dev
- Groq API — console.groq.com
- Ollama — ollama.com
- Streamlit — docs.streamlit.io
- python-dotenv (Python) — pypi.org/project/python-dotenv
- dotenv (JavaScript/Node.js) — npmjs.com/package/dotenv
- C++ HTTP/REST — libcurl (curl.se/libcurl) or cpp-httplib (github.com/yhirose/cpp-httplib); for environment variables, see dotenv-cpp (github.com/adeharo9/cpp-dotenv)
- GitHub documentation — docs.github.com

From the programme team:

The capstone is not designed to catch you out. It is designed to give you something concrete to walk away with — a project you built, can explain, and can put in front of an employer.

Pick something that genuinely interests you, keep the scope tight, and focus on getting the core working well before adding anything extra. A clean, working application presented with confidence is exactly what we are looking for.